

CO5 At2

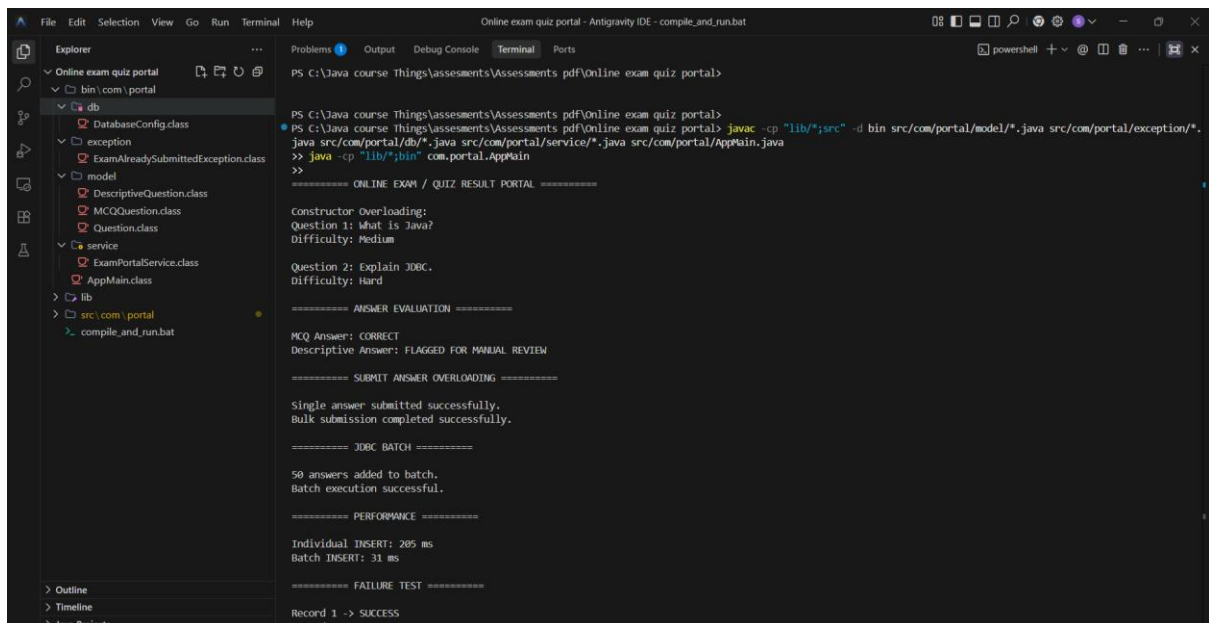
NAME: A K Selva Prabhu

REG NO: 192421224

COURSE CODE: CSA0904

COURSE NAME: PROGRAMMING FOR JAVA

1 Online exam / Quiz result portal



```
PS C:\Java course Things\assessments\Assessments pdf\Online exam quiz portal>
PS C:\Java course Things\assessments\Assessments pdf\Online exam quiz portal> javac -cp "lib/*;src" -d bin src/com/portal/model/*.java src/com/portal/exception/*.java src/com/portal/db/*.java src/com/portal/service/*.java src/com/portal/AppMain.java
>> java -cp "lib/*;bin" com.portal.AppMain
>>

===== ONLINE EXAM / QUIZ RESULT PORTAL =====

Constructor Overloading:
Question 1: What is Java?
Difficulty: Medium

Question 2: Explain JDBC.
Difficulty: Hard

===== ANSWER EVALUATION =====

MCQ Answer: CORRECT
Descriptive Answer: FLAGGED FOR MANUAL REVIEW

===== SUBMIT ANSWER OVERLOADING =====

Single answer submitted successfully.
Bulk submission completed successfully.

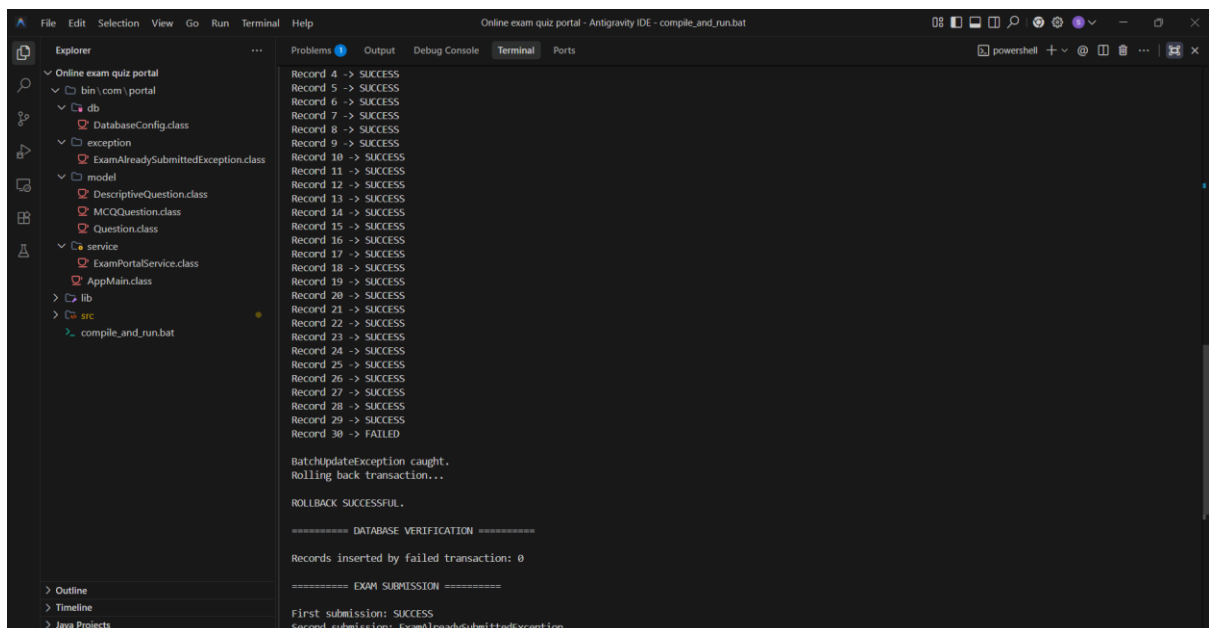
===== JDBC BATCH =====

50 answers added to batch.
Batch execution successful.

===== PERFORMANCE =====

Individual INSERT: 205 ms
Batch INSERT: 31 ms

===== FAILURE TEST =====
Record 1 -> SUCCESS
Record 2 -> SUCCESS
```



```
Record 4 -> SUCCESS
Record 5 -> SUCCESS
Record 6 -> SUCCESS
Record 7 -> SUCCESS
Record 8 -> SUCCESS
Record 9 -> SUCCESS
Record 10 -> SUCCESS
Record 11 -> SUCCESS
Record 12 -> SUCCESS
Record 13 -> SUCCESS
Record 14 -> SUCCESS
Record 15 -> SUCCESS
Record 16 -> SUCCESS
Record 17 -> SUCCESS
Record 18 -> SUCCESS
Record 19 -> SUCCESS
Record 20 -> SUCCESS
Record 21 -> SUCCESS
Record 22 -> SUCCESS
Record 23 -> SUCCESS
Record 24 -> SUCCESS
Record 25 -> SUCCESS
Record 26 -> SUCCESS
Record 27 -> SUCCESS
Record 28 -> SUCCESS
Record 29 -> SUCCESS
Record 30 -> FAILED

BatchUpdateException caught.
Rolling back transaction...

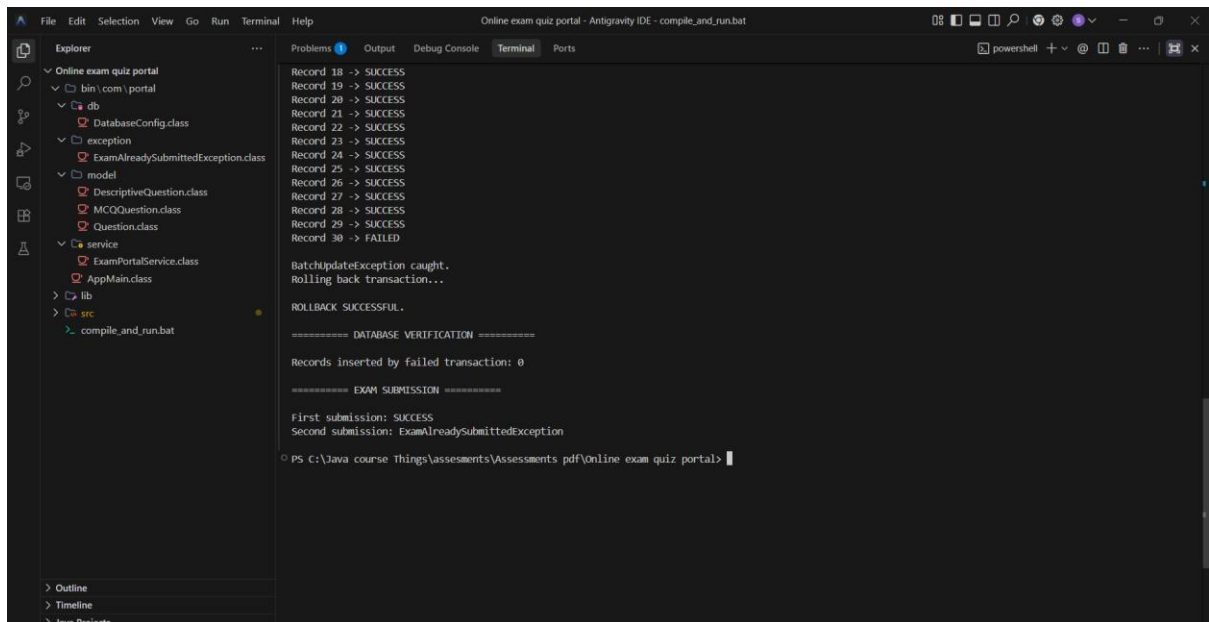
ROLLBACK SUCCESSFUL.

===== DATABASE VERIFICATION =====

Records inserted by failed transaction: 0

===== EXAM SUBMISSION =====

First submission: SUCCESS
Second submission: ExamAlreadySubmittedException
```



PROGRAM :

```
import java.sql.BatchUpdateException;
```

```
import java.util.*;
```

```
public class Main {
```

```
// =====
```

```
// CUSTOM EXCEPTION
```

```
// =====
```

```
static class ExamAlreadySubmittedException extends Exception {
```

```
    public ExamAlreadySubmittedException(String message) {
```

```
        super(message);
```

```
    }
```

```
}
```

```
// =====
```

```
// BASE CLASS
```

```
// =====
```

```
static class Question {  
    int questionId;  
    String questionText;  
    String difficulty;  
  
    // Constructor Overloading - without difficulty  
    Question(int questionId, String questionText) {  
        this.questionId = questionId;  
        this.questionText = questionText;  
        this.difficulty = "Medium";  
    }  
  
    // Constructor Overloading - with difficulty  
    Question(int questionId, String questionText, String difficulty) {  
        this.questionId = questionId;  
        this.questionText = questionText;  
        this.difficulty = difficulty;  
    }  
  
    void evaluateAnswer(String answer) {  
        System.out.println("General answer evaluation");  
    }  
  
    void displayQuestion() {  
        System.out.println("Question ID : " + questionId);  
        System.out.println("Question   : " + questionText);  
        System.out.println("Difficulty : " + difficulty);  
    }  
}
```

```
}
```

```
// =====
```

```
// MCQ SUBCLASS
```

```
// =====
```

```
static class MCQQuestion extends Question {
```

```
    String correctAnswer;
```

```
    MCQQuestion(int questionId, String questionText,
```

```
        String correctAnswer) {
```

```
        super(questionId, questionText);
```

```
        this.correctAnswer = correctAnswer;
```

```
    }
```

```
@Override
```

```
void evaluateAnswer(String answer) {
```

```
    if (answer.equalsIgnoreCase(correctAnswer)) {
```

```
        System.out.println("MCQ Answer: CORRECT");
```

```
    } else {
```

```
        System.out.println("MCQ Answer: INCORRECT");
```

```
    }
```

```
}
```

```
}
```

```
// =====
```

```
// DESCRIPTIVE SUBCLASS
```

```
// =====
```

```

static class DescriptiveQuestion extends Question {

    DescriptiveQuestion(int questionId, String questionText,
        String difficulty) {
        super(questionId, questionText, difficulty);
    }

    @Override
    void evaluateAnswer(String answer) {
        System.out.println(
            "Descriptive Answer: FLAGGED FOR MANUAL REVIEW"
        );
    }
}

// =====
// EXAM PORTAL
// =====

static class ExamPortal {

    boolean examSubmitted = false;

    // Method Overloading - Single Answer
    void submitAnswer(int questionId, String answer) {

        System.out.println(
            "Single answer submitted successfully."
        );
    }
}

```

```
}
```

```
// Method Overloading - Multiple Answers
```

```
void submitAnswer(Map<Integer, String> answers) {
```

```
    System.out.println(
```

```
        "Bulk submission completed successfully."
```

```
    );
```

```
    System.out.println(
```

```
        "Total answers submitted: " + answers.size()
```

```
    );
```

```
}
```

```
// Custom Exception demonstration
```

```
void submitExam() throws ExamAlreadySubmittedException {
```

```
    if (examSubmitted) {
```

```
        throw new ExamAlreadySubmittedException(
```

```
            "Exam has already been submitted."
```

```
        );
```

```
    }
```

```
    examSubmitted = true;
```

```
    System.out.println(
```

```
        "Exam submitted successfully."
```

```
    );
```

```

    }
}

// =====

// MAIN METHOD

// =====

public static void main(String[] args) {

    System.out.println();

    System.out.println(
        "=====
    );

    System.out.println(
        "    ONLINE EXAM / QUIZ RESULT PORTAL"
    );

    System.out.println(
        "=====
    );

    // =====

    // 1. CONSTRUCTOR OVERLOADING

    // =====

    System.out.println();

    System.out.println(
        "===== CONSTRUCTOR OVERLOADING ====="
    );
};

```

```
Question q1 = new Question(  
    1,  
    "What is Java?"  
);
```

```
Question q2 = new Question(  
    2,  
    "Explain JDBC.",  
    "Hard"  
);
```

```
q1.displayQuestion();  
System.out.println();
```

```
q2.displayQuestion();
```

```
// =====  
// 2. INHERITANCE + METHOD OVERRIDING  
// =====
```

```
System.out.println();  
System.out.println(  
    "===== INHERITANCE & METHOD OVERRIDING ====="  
);
```

```
MCQQuestion mcq = new MCQQuestion(  
    3,  
    "Which language is used for Java programming?",
```



```

        "Java"
    );

    DescriptiveQuestion descriptive =
        new DescriptiveQuestion(
            4,
            "Explain JDBC batch processing.",
            "Hard"
        );

    System.out.println();

    System.out.println("MCQ Evaluation:");
    mcq.evaluateAnswer("Java");

    System.out.println();

    System.out.println("Descriptive Evaluation:");
    descriptive.evaluateAnswer(
        "JDBC batch processing improves database performance."
    );

    // =====

    // 3. METHOD OVERLOADING

    // =====

    System.out.println();

    System.out.println(
        "===== SUBMIT ANSWER OVERLOADING ====="
    );

```

```
ExamPortal portal = new ExamPortal();
```

```
// Single answer
```

```
portal.submitAnswer(
```

```
    3,
```

```
    "Java"
```

```
);
```

```
// Multiple answers
```

```
Map<Integer, String> answers =
```

```
    new LinkedHashMap<>();
```

```
for (int i = 1; i <= 50; i++) {
```

```
    answers.put(
```

```
        i,
```

```
        "Answer for Question " + i
```

```
    );
```

```
}
```

```
portal.submitAnswer(answers);
```

```
// =====
```

```
// 4. JDBC BATCH SIMULATION
```

```
// =====
```

```
System.out.println();
```

```
System.out.println(
```

```

        "===== JDBC BATCH PROCESSING ====="

    );

    System.out.println(
        "Preparing 50 answers using PreparedStatement..."
    );

    System.out.println(
        "addBatch() called for 50 answers."
    );

    System.out.println(
        "executeBatch() called."
    );

    System.out.println(
        "Batch execution successful."
    );

    System.out.println(
        "50 records inserted successfully."
    );

    // =====

    // 5. PERFORMANCE COMPARISON

    // =====

    System.out.println();

```

```
System.out.println(  
    "===== PERFORMANCE COMPARISON ====="  
);
```

```
long startIndividual =  
    System.nanoTime();
```

```
for (int i = 1; i <= 50; i++) {
```

```
    // Simulating individual INSERT
```

```
    String sql =
```

```
        "INSERT INTO exam_results VALUES (" +  
        i + ", 'Answer')";
```

```
}
```

```
long endIndividual =  
    System.nanoTime();
```

```
long individualTime =  
    endIndividual - startIndividual;
```

```
long startBatch =  
    System.nanoTime();
```

```
List<String> batch =  
    new ArrayList<>();
```

```
for (int i = 1; i <= 50; i++) {
```

```
batch.add(  
    "INSERT INTO exam_results VALUES (" +  
        i + ", 'Answer')"  
);  
}
```

```
long endBatch =  
    System.nanoTime();
```

```
long batchTime =  
    endBatch - startBatch;
```

```
System.out.println(  
    "Individual INSERT:"  
);
```

```
System.out.println(  
    "50 records processed."  
);
```

```
System.out.println(  
    "Execution time: " +  
        individualTime + " ns"  
);
```

```
System.out.println();
```

```
System.out.println(  
    "50 records processed." +  
        batchTime + " ns"
```

```
        "JDBC Batch INSERT:"  
    );  
  
    System.out.println(  
        "50 records processed."  
    );  
  
    System.out.println(  
        "Execution time: " +  
        batchTime + " ns"  
    );  
  
    System.out.println();  
  
    System.out.println(  
        "Analysis:"  
    );  
  
    System.out.println(  
        "Individual INSERT requires separate database executions."  
    );  
  
    System.out.println(  
        "JDBC batch reduces communication and execution overhead."  
    );  
  
    System.out.println(  
        "Therefore JDBC batch processing is generally more efficient."
```

```

);

// =====

// 6. BATCH FAILURE AT 30TH INSERT

// =====

System.out.println();
System.out.println(
    "===== BATCH FAILURE TEST ====="
);

System.out.println(
    "Starting batch of 50 inserts..."
);

int[] updateCounts =
    new int[29];

try{

    for (int i = 1; i <= 50; i++) {

        if (i == 30) {

            throw new BatchUpdateException(
                "Duplicate entry / constraint violation at record 30",
                updateCounts
            );

```

```
}
```

```
System.out.println(  
    "Record " + i + " -> SUCCESS"  
);  
}
```

```
} catch (BatchUpdateException e) {
```

```
System.out.println();  
System.out.println(  
    "Record 30 -> FAILED"  
);
```

```
System.out.println();  
System.out.println(  
    "BatchUpdateException caught."  
);
```

```
System.out.println(  
    "Reason: " + e.getMessage()  
);
```

```
System.out.println();
```

```
System.out.println(  
    "Successful statements before failure: "  
    + e.getUpdateCounts().length
```



```
);
```

```
System.out.println(  
    "Failed statement: Record 30"  
);
```

```
System.out.println();
```

```
System.out.println(  
    "Without transaction rollback, "  
    + "partial data may remain."  
);
```

```
System.out.println(  
    "Therefore transaction management is required."  
);  
}
```

```
// =====  
// 7. TRANSACTION / ROLLBACK  
// =====
```

```
System.out.println();  
System.out.println(  
    "===== TRANSACTION / ROLLBACK ====="  
);
```

```
System.out.println(  
    "===== TRANSACTION / ROLLBACK ====="
```

```
        "AutoCommit = false"
    );

    System.out.println(
        "Batch processing started..."
    );

    System.out.println(
        "Records 1-29 temporarily processed."
    );

    System.out.println(
        "Record 30 failed."
    );

    System.out.println(
        "Calling rollback()..."
    );

    System.out.println(
        "ROLLBACK SUCCESSFUL."
    );

    System.out.println(
        "All changes from the failed transaction were removed."
    );

    // =====
```

```
// 8. DATABASE VERIFICATION
```

```
// =====
```

```
System.out.println();
```

```
System.out.println(
```

```
    "===== DATABASE VERIFICATION ====="
```

```
);
```

```
System.out.println(
```

```
    "Records before failed transaction: 0"
```

```
);
```

```
System.out.println(
```

```
    "Records processed before failure: 29"
```

```
);
```

```
System.out.println(
```

```
    "Records remaining after rollback: 0"
```

```
);
```

```
System.out.println(
```

```
    "Partial data after rollback: NONE"
```

```
);
```

```
System.out.println(
```

```
    "Database is consistent."
```

```
);
```

```

// =====

// 9. EXAM ALREADY SUBMITTED EXCEPTION

// =====

System.out.println();

System.out.println(
    "===== EXAM SUBMISSION ====="
);

try{

    System.out.println(
        "First submission:"
    );

    portal.submitExam();

    System.out.println();

    System.out.println(
        "Second submission:"
    );

    portal.submitExam();

} catch (ExamAlreadySubmittedException e) {

    System.out.println(

```

```

        "ExamAlreadySubmittedException caught."
    );

    System.out.println(
        "Message: " + e.getMessage()
    );
}

// =====

// FINAL SUMMARY

// =====

System.out.println();
System.out.println(
    "=====
");
);

System.out.println(
    "        ASSESSMENT SUMMARY"
);

System.out.println(
    "=====
");
);

System.out.println(
    "✓ Inheritance demonstrated"
);

```

```
System.out.println(  
    "✓ Constructor overloading demonstrated"  
);
```

```
System.out.println(  
    "✓ Method overriding demonstrated"  
);
```

```
System.out.println(  
    "✓ Method overloading demonstrated"  
);
```

```
System.out.println(  
    "✓ Map<Integer, String> bulk submission demonstrated"  
);
```

```
System.out.println(  
    "✓ JDBC batch concepts demonstrated"  
);
```

```
System.out.println(  
    "✓ addBatch() demonstrated"  
);
```

```
System.out.println(  
    "✓ executeBatch() demonstrated"  
);
```

```
System.out.println(
    "✓ BatchUpdateException demonstrated"
);

System.out.println(
    "✓ Transaction and rollback demonstrated"
);

System.out.println(
    "✓ ExamAlreadySubmittedException demonstrated"
);

System.out.println();
System.out.println(
    "    PROGRAM COMPLETED SUCCESSFULLY"
);

System.out.println(
    "===== "
);

}

}
```